CUDA-Accelerated Multi-Exposure Image Fusion

Team

Estella Xie (ziyoux), Junhao Li (junhaoli)

Project URL

https://Lijunhaohz.github.io/15618-project/

Summary

We propose to implement and optimize a CUDA-based multi-exposure image fusion pipeline that combines photographs of the same scene taken at different exposure levels into a single image with well-preserved details in both bright and dark regions. Beyond implementing the multi-scale fusion pipeline, we will develop a memory-aware execution planner that selects between full-resident and incremental processing under a specified GPU memory budget. We will investigate selective kernel fusion, shared-memory tiling, and intermediate-buffer reuse, evaluating their trade-offs in execution time, recomputation, data transfer, and peak GPU memory usage.

Background

Photographs captured at a single exposure often lose details in bright highlights or dark shadows. **Multi-exposure fusion** combines complementary information from differently exposed photographs into a single displayable image. We will follow the Exposure Fusion algorithm proposed by Mertens et al., assuming pre-aligned RGB images of a static scene.

The algorithm consists of four main stages:

1. **Weight computation:** Compute per-pixel weights based on contrast, saturation, and well-exposedness, then normalize weights across input images.
2. **Pyramid construction:** Build Gaussian pyramids of the weight maps and Laplacian pyramids of the input images using filtering and downsampling.
3. **Multi-scale blending:** Compute weighted combinations of the Laplacian pyramids at each level.
4. **Reconstruction:** Progressively upsample and combine pyramid levels to produce the final image.

The main data structures are full-resolution image and weight arrays, per-image pyramids, and an accumulated output pyramid. Within each level, weight calculations, filtering, and blending expose substantial data parallelism.

Our project will use this pipeline to investigate how selective kernel fusion, intermediate-buffer reuse, and execution order affect latency and peak GPU memory consumption. We will first establish a CPU reference and a separate-kernel CUDA baseline, then compare optimized execution strategies while preserving the same algorithm and boundary rules.

The Challenge

Although exposure fusion offers substantial pixel-level parallelism, efficient GPU execution requires coordinating spatial, cross-image, and cross-scale dependencies. Neighborhood filters require overlapping input regions, weight normalization depends on all exposures, and pyramid construction and reconstruction impose dependencies between levels.

Our main target is to reduce data movement and memory consumption without introducing excessive recomputation, synchronization, or GPU resource usage.

1. Dependency-Aware Kernel Fusion

Although individual pixel operations are highly parallel, dependencies across images, neighboring pixels, and pyramid levels introduce synchronization and memory overhead. For example, Gaussian/Laplacian pyramid construction and reconstruction introduce sequential dependencies between levels. Each thread block must load sufficient halo data and correctly handle intermediate boundaries. Larger fused regions may introduce redundant computation and increase register or shared-memory usage, reducing occupancy. We will investigate which fusion boundaries provide useful data reuse and when separate execution is preferable.

2. Execution Order and Memory-Capacity Trade-offs

The pyramid building has two different implementation strategies: full-resident and incremental.

A **full-resident implementation** retains inputs and required per-image intermediates in GPU memory, enabling direct reuse but increasing memory consumption with exposure count and resolution. An **incremental implementation** processes exposures individually and accumulates their pyramid contributions, reducing resident storage. However, normalized weights require the weight sum across all exposures, preventing a straightforward single-pass approach.

We will compare retaining intermediate data with a multi-pass strategy that recomputes or reloads weights, evaluating the trade-off between peak memory usage, additional computation, data transfer, and total execution time

3. Uneven Parallelism Across Pyramid Levels

Image dimensions shrink at each pyramid level, creating different GPU utilization challenges. Fine levels offer abundant parallelism but high memory traffic, while coarse levels may underutilize the GPU. Dependencies between successive levels also restrict scheduling freedom, even though separate kernel launches in the same stream can enforce execution order without CPU synchronization. We will investigate level-specific kernel configurations to improve performance across different workload sizes.

4. Memory Access Patterns and Data Reuse

The pipeline repeatedly accesses large images and intermediate pyramids, with relatively little computation per memory access, making it potentially memory-bandwidth-bound. **Neighborhood filtering** exhibits spatial locality, but naive implementations repeatedly load overlapping pixels from global memory. We will explore kernel fusion and shared-memory tiling to reduce memory traffic, while balancing register pressure, shared-memory usage, and GPU occupancy.

Reducing peak memory usage also requires careful management of intermediate dependencies: **buffers can be reused** only after their contents are no longer needed by filtering, pyramid construction, or

reconstruction. For our streaming extension, cross-image weight normalization introduces an additional constraint, since each exposure's normalized weights depend on all input exposures. Incremental processing may therefore require multiple passes, trading lower memory consumption for recomputation or additional transfers. Throughout these optimizations, we must preserve normalization, sampling, and boundary conventions, validating intermediate results against a CPU reference to distinguish acceptable floating-point differences from implementation errors.

To summarize, our central concern is:

> How should computation and intermediate storage be organized to balance execution time and peak GPU memory usage while preserving the exposure-fusion algorithm?

Through this project, we aim to understand how **algorithmic dependencies, memory bandwidth, and GPU resource constraints** affect performance, and how to balance execution speed, parallelism, and memory efficiency through CUDA optimizations.

Resources

We will develop the core CPU reference and CUDA implementations from scratch. Existing libraries will be used for image loading and saving, rather than for the core GPU fusion pipeline.

Planned resources include:

- NVIDIA RTX 2080 GPUs on the GHC machines.
- CUDA development tools, CUDA-event timing, and available NVIDIA profiling tools.
- Pre-aligned exposure brackets from public datasets and/or controlled tripod captures.
- Synthetic constant images, impulses, and odd-sized images for boundary and reconstruction checks. No special hardware beyond a CUDA-capable GPU is initially required.

Goals and Deliverables

50% Target — Functional Baseline

- Implement a CPU reference and a separate-kernel CUDA pipeline covering weight computation, normalization, pyramid construction, blending, and reconstruction.
- Produce fused images from pre-aligned exposures and verify correctness against the CPU reference.

100% Target — Optimized GPU Pipeline

- Optimize neighborhood operations using shared-memory tiling and implement at least one neighborhood-based fusion strategy, such as filtering with downsampling.
- Reuse intermediate buffers based on their lifetimes to reduce peak GPU memory consumption.
- Implement both full-resident and multi-pass incremental execution. The incremental mode will preserve cross-image weight normalization while avoiding simultaneous storage of all input pyramids.
- Compare the CPU reference, baseline CUDA pipeline, and optimized implementation using execution time, stage-level timings, peak memory usage, and numerical error.

- Deliver a command-line application, representative visual results, and a performance evaluation explaining the benefits and limitations of each optimization.

150% Target — Memory-Aware Incremental Execution

- Develop a memory-aware planner that estimates storage requirements and automatically selects a feasible execution mode under a specified GPU memory budget.
- Use performance measurements to guide mode selection when both modes fit within the budget.
- Overlap host–device transfers with computation using CUDA streams and double buffering, ensuring correct dependencies and safe buffer reuse.
- Evaluate planning decisions and transfer overlap against the manually selected, synchronous implementations.

Evaluation

We will compare the CPU reference, separate-kernel CUDA baseline, and optimized implementations using identical algorithm parameters, sampling conventions, and boundary rules.

Our evaluation will include:

- **Correctness**: Compare intermediate and final floating-point outputs against the CPU reference using maximum absolute error and RMSE. Visual examples will demonstrate fusion quality separately from numerical correctness.
- **Performance**: Measure GPU pipeline latency and processing time including host–device transfers. Report stage-level timings to identify bottlenecks.
- **Memory efficiency**: Measure peak device-memory allocation and compare full-resident and incremental execution, quantifying memory savings alongside additional recomputation, transfers, and execution time.
- **Optimization ablations**: Evaluate tiling, kernel fusion, and buffer reuse individually and in combination to identify their contributions and interactions.

Our experiments will vary image resolution, exposure count, and pyramid depth. Profiling will be used to investigate whether performance is limited by memory traffic, launch overhead, synchronization, or insufficient parallelism.

For the 150% goal, we will additionally compare automatic planning with manually selected execution modes and asynchronous execution with its synchronous counterpart.

Platform Choice

We will use C++/CUDA on the NVIDIA RTX 2080 GPUs in the GHC machines. Exposure fusion offers substantial pixel-level parallelism in weight computation, filtering, and blending. CUDA provides explicit control over shared memory, kernel fusion, and buffer allocation, making it suitable for exploring execution-time and memory trade-offs between full-resident and incremental processing.

Schedule

Dates	Planned Work
Nov 8–13	Finalize algorithm and boundary conventions, prepare input datasets, and implement the CPU reference.
Nov 14–18	Complete the separate-kernel CUDA baseline and validate intermediate stages and final outputs. Profile the baseline to identify performance bottlenecks.
Nov 19–23	Analyze profiling results, obtain preliminary optimization results, and submit the **Nov 23 milestone report** with an updated schedule. Implement more optimization strategies if ahead of schedule.
Nov 24–30	Combine shared-memory tiling, selected neighborhood fusion, buffer reuse, and a correct multi-pass incremental execution mode. Run experiments to evaluate the combined optimizations.
Dec 1–4	Compare full-resident and incremental execution; run correctness checks and optimization ablations across workload sizes. Attempt automatic planning and transfer overlap if ahead of schedule.
Dec 5–10	Complete experiments and performance analysis, finalize deliverables, and submit the **Dec 10 final report**.
Dec 11	**Poster session**: present visual results, performance findings, and memory–runtime trade-offs.

Proposed division:

- **Estella**: Lead quality-weight computation, cross-image normalization, and weighted pyramid accumulation. Implement the corresponding CPU and CUDA stages and the multi-pass incremental processing workflow.
- **Junhao**: Lead Gaussian/Laplacian pyramid construction and image reconstruction. Implement the corresponding CPU and CUDA stages, shared-memory tiling, and filtering/downsampling fusion.

Both members will collaborate on pipeline integration, buffer reuse, correctness validation, profiling, and performance experiments. Stretch goals will be divided based on progress.

Reference

Tom Mertens, Jan Kautz, and Frank Van Reeth. Exposure Fusion. Pacific Graphics, 2007.